

Predictor + QMAR 技术实现说明

面向 Claude Code 的系统模型、问题定义与算法设计

Edge VLM Routing / MASS Draft

2026-07-01

本说明用于指导 Claude Code 实现一个可实验复现的 Predictor + QMAR pipeline。目标不是一次性实现完整在线系统，而是先跑通：数据适配、离线 profiling、multi-task multimodal predictor、answer-type-aware latency estimation、QMAR batch scheduler 与实验评估。

Contents

1	实现目标与边界	3
1.1	总体目标	3
1.2	实现边界	3
2	数据使用方案	3
2.1	推荐数据集角色	3
2.2	统一数据 schema	3
2.3	数据标签	4
2.3.1	Suitability label	4
2.3.2	Answer type / length class label	4
3	System Model	4
3.1	Request batch	4
3.2	Model pool and instance pool	5
3.3	Quality and latency separation	5
3.4	Quality threshold	5
4	Offline Profiling	5
4.1	Quality profile	5
4.2	Latency profile	6
4.3	Communication profile	6
5	Predictor 模块设计	6
5.1	模块定位	6
5.2	输入输出	6
5.3	推荐结构	7
5.4	Vision encoder	7
5.5	Text encoder	7
5.6	Fusion	7
5.7	Suitability head	7
5.8	Answer-type head	8

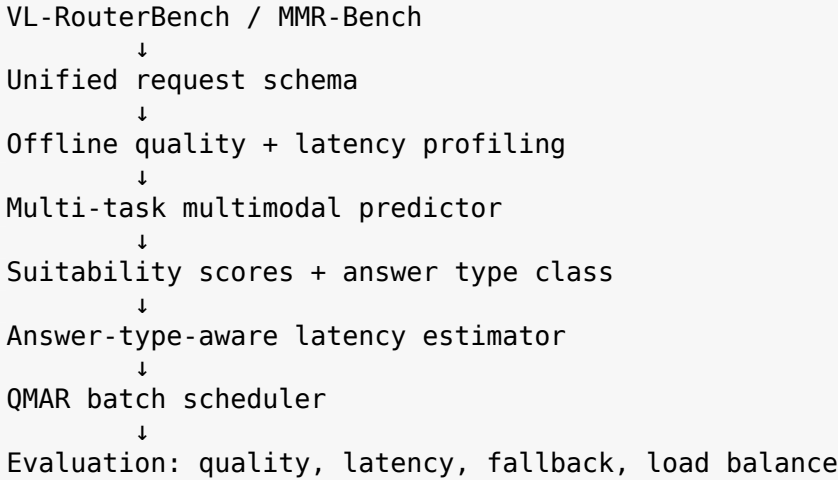
6 Request-specific Latency Estimation	8
6.1 Latency definition	8
6.2 可选细化公式	8
7 Problem Formulation	8
7.1 Quality feasible set	8
7.2 Assignment variable	8
7.3 Instance load and objective	9
7.4 问题类型	9
8 QMAR 算法设计	9
8.1 算法定位	9
8.2 算法输入	9
8.3 Step 1: Feasible set construction	10
8.4 Step 2: Request ranking	10
8.4.1 Feasible set size	10
8.4.2 Quality margin	10
8.4.3 Minimum feasible latency	10
8.4.4 Lexicographic sorting rule	10
8.5 Step 3: Load-balanced greedy assignment	11
8.6 Fallback rule	11
8.7 Pseudocode	11
8.8 Complexity	12
9 Implementation Components for Claude Code	12
9.1 建议模块划分	12
9.2 推荐目录结构	12
10 Baselines and Ablations	13
10.1 必要 baselines	13
10.2 可选 oracle baseline	13
11 Evaluation Metrics	13
11.1 Quality-side metrics	14
11.2 System-side metrics	14
11.3 Routing-side metrics	14
12 Recommended Experiment Sequence	14
12.1 Phase 1: VL-RouterBench main pipeline	14
12.2 Phase 2: MMR-Bench validation	15
12.3 Phase 3: Ablation	15
13 Expected Outputs	15
14 Implementation Notes	15
14.1 关键设计约束	15
14.2 最终一句话定位	16

1 实现目标与边界

1.1 总体目标

需要实现一个面向 VLM routing 的 batch-level scheduler。系统接收一批 image-text requests, 预测每个 request 对候选 VLM 的质量适配度, 并结合回答类型估计 request-instance latency, 最终在质量阈值约束下最小化 batch latency。

核心 pipeline 如下:



1.2 实现边界

第一版实现应保持轻量, 不需要覆盖完整在线部署系统。

- 不需要实现复杂 cross-attention fusion。
- 不需要在线更新 quality predictor。
- 不需要精确预测 output length, 只需要预测 answer type 或 length class。
- 不需要把 QMAR 写成 ILP 求解器; ILP 只可作为小规模 oracle baseline。
- 可以先做 simulator, 再做少量真实 edge runtime validation。

2 数据使用方案

2.1 推荐数据集角色

可以使用 VL-RouterBench 与 MMR-Bench, 但不要在第一版中直接混合训练。推荐定位如下:

数据集	推荐角色
VL-RouterBench	主实验数据集。用于 main training、main evaluation、完整 Predictor + QMAR pipeline。
MMR-Bench	外部验证数据集。用于 robustness、cross-benchmark validation、protocol sensitivity analysis。
两者合并	第一版不推荐。除非统一 candidate model pool、quality label protocol 和 latency profile。

2.2 统一数据 schema

无论数据来自 VL-RouterBench 还是 MMR-Bench, 进入系统后都应转成统一 request-level schema。

```

RequestRecord:
  request_id
  dataset_name
  task_name / subset_name
  image_path or image_id
  question_text
  ground_truth_answer
  candidate_model_outputs
  candidate_model_correctness
  optional_original_utility
  optional_task_metric_label

```

其中 candidate_model_correctness 用于构造 suitability label。若 benchmark 只提供 utility 或 normalized score，需要先转成二值 correctness 或 thresholded quality label。

2.3 数据标签

Predictor 需要两类标签。

2.3.1 Suitability label

对每个 request r_i 与候选模型 m ，构造：

$$y_{i,m} \in \{0, 1\}$$

其中 $y_{i,m} = 1$ 表示模型 m 对 request r_i 的回答满足任务质量标准。

2.3.2 Answer type / length class label

第一版建议使用 length bucket，而不是复杂语义分类：

Class	Criterion	Meaning
short	≤ 5 tokens	yes/no, number, color, object name
medium	6–30 tokens	OCR phrase, short description
long	> 30 tokens	explanation, reasoning, detailed description

标签优先从 ground-truth answer 或 reference answer 生成。若 ground-truth 很短但问题明显需要推理，可以在后续版本加入 semantic type 规则。

3 System Model

3.1 Request batch

系统处理一个 batch：

$$\mathcal{B} = \{r_1, r_2, \dots, r_N\}$$

每个 request 是一个 image-text pair：

$$r_i = (I_i, T_i)$$

其中 I_i 是图像， T_i 是文本问题。

3.2 Model pool and instance pool

候选 VLM 模型集合为：

$$\mathcal{M} = \{m_1, m_2, \dots, m_M\}$$

边缘实例池为：

$$\mathcal{U} = \{u_1, u_2, \dots, u_K\}$$

其中每个 instance 表示一个模型在某个节点上的部署：

$$u_k = (m(k), d(k))$$

$m(k)$ 表示 instance u_k 部署的 VLM, $d(k)$ 表示部署节点。

3.3 Quality and latency separation

实现时必须区分 model-side quality 与 instance-side latency。

- Suitability 是 model-side quality, 与模型能力相关。
 - Latency 是 instance-side system cost, 与模型、设备、通信、负载和回答类型相关。
- 若多个 instance 部署同一个 VLM, 则它们共享 suitability score:

$$s_{i,k} = s_{i,m(k)}$$

但它们有不同 latency:

$$l_{i,k} \neq l_{i,k'} \quad \text{if } d(k) \neq d(k')$$

3.4 Quality threshold

每个 request 可以有一个质量阈值：

$$\tau_i$$

第一版也可以使用全局阈值 τ 。为了保持 generality, 推荐代码接口保留 request-specific threshold。

4 Offline Profiling

4.1 Quality profile

对每个 request 与每个候选 VLM, 记录模型是否回答正确：

$$y_{i,m} \in \{0, 1\}$$

输出表建议包含：

```
quality_profile:
  request_id
  model_name
  model_output
  correctness_label
  optional_score
  evaluation_protocol
```

4.2 Latency profile

QMAR 需要 request-instance latency。benchmark 通常不直接提供真实 edge latency，因此需要额外 profile 或模拟。

第一版推荐建立 answer-type-based latency table：

$$\bar{l}_{k,a}$$

表示 instance u_k 处理 answer type a 的平均 latency。

Instance	short	medium	long
VLM-3B@Orin	$\bar{l}_{1,short}$	$\bar{l}_{1,medium}$	$\bar{l}_{1,long}$
VLM-7B@3090	$\bar{l}_{2,short}$	$\bar{l}_{2,medium}$	$\bar{l}_{2,long}$
VLM-13B@4090	$\bar{l}_{3,short}$	$\bar{l}_{3,medium}$	$\bar{l}_{3,long}$

4.3 Communication profile

通信开销可以先用简单模型估计：

$$l_{i,k}^{comm} = RTT_k + \frac{size(I_i) + size(T_i)}{BW_k}$$

如果实验环境暂时没有真实网络数据，可以使用 synthetic RTT 与 bandwidth profile。

5 Predictor 模块设计

5.1 模块定位

Predictor 不是单纯的 visual-only quality predictor，而是 multi-task multimodal request predictor。它同时输出：

- model suitability scores，用于 quality feasibility 判断；
- answer type / length class，用于 request-specific latency estimation。

5.2 输入输出

输入为 request：

$$r_i = (I_i, T_i)$$

输出为：

$$\mathbf{s}_i = [s_{i,1}, s_{i,2}, \dots, s_{i,M}]$$

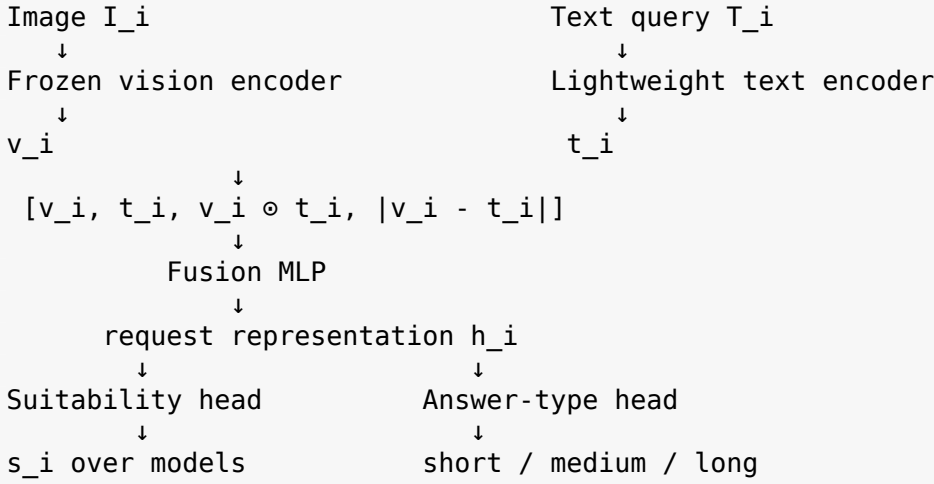
以及：

$$\hat{a}_i \in \{short, medium, long\}$$

注意 $\mathbf{s}_i$ 的维度建议对应 candidate models，而不是 instances。运行时再根据 $m(k)$ 映射到 instance。

5.3 推荐结构

第一版采用 frozen encoders + lightweight fusion MLP。



5.4 Vision encoder

可选 CLIP、SigLIP、EVA 或 VLM vision tower。第一版建议 frozen，不做端到端 fine-tuning。它负责提取 visual complexity、OCR difficulty、layout、small text、spatial relation 等特征。

5.5 Text encoder

使用 lightweight text encoder，例如 MiniLM、DistilBERT、TinyBERT 或 BERT-small。它负责捕捉 question intent，例如 OCR、counting、yes/no、description、reasoning、chart/document understanding 等。

5.6 Fusion

推荐使用轻量 MLP fusion：

$$z_i = [v_i, t_i, v_i \odot t_i, |v_i - t_i|]$$

$$h_i = MLP(z_i)$$

该方案足够表达 image-question pair interaction，且实现复杂度低。

5.7 Suitability head

Suitability head 输出每个候选模型的 suitability score：

$$\mathbf{s}_i = \sigma(W_s h_i + b_s)$$

训练目标是 multi-label binary classification：

$$\mathcal{L}_{suit} = \sum_{m=1}^M BCE(s_{i,m}, y_{i,m})$$

如果正负样本不平衡，可以替换为 focal BCE。

5.8 Answer-type head

Answer-type head 输出 answer type / workload class:

$$\hat{a}_i = \text{softmax}(W_a h_i + b_a)$$

训练 loss:

$$\mathcal{L}_{type} = CE(\hat{a}_i, a_i)$$

总 loss:

$$\mathcal{L} = \mathcal{L}_{suit} + \alpha \mathcal{L}_{type}$$

其中 α 可在 validation set 上调整。

6 Request-specific Latency Estimation

6.1 Latency definition

令 $l_{i,k}$ 表示 request r_i 分配给 instance u_k 的 expected latency。

第一版建议使用:

$$l_{i,k} = \bar{l}_{k,\hat{a}_i} + l_{i,k}^{comm}$$

其中 $\bar{l}_{k,\hat{a}_i}$ 来自 instance latency table, $l_{i,k}^{comm}$ 来自 communication profile。

6.2 可选细化公式

若需要更细粒度解释, 可以写成:

$$l_{i,k} = l_{i,k}^{comm} + l_k^{base} + g(\hat{a}_i) \cdot l_k^{dec}$$

其中 $g(\hat{a}_i)$ 是 answer type 对应的 workload factor, l_k^{dec} 是 instance 单位生成开销。

7 Problem Formulation

7.1 Quality feasible set

对每个 request, 先根据 quality threshold 过滤可行 instance:

$$\mathcal{F}_i = \{u_k \in \mathcal{U} \mid s_{i,m(k)} \geq \tau_i\}$$

只有 $\mathcal{F}_i$ 中的 instance 可以作为正常 assignment candidate。

7.2 Assignment variable

定义:

$$x_{i,k} = \begin{cases} 1, & \text{if request } r_i \text{ is assigned to instance } u_k, \\ 0, & \text{otherwise.} \end{cases}$$

每个 request 只能分配给一个 instance:

$$\sum_{k=1}^K x_{i,k} = 1, \quad \forall i$$

quality feasibility constraint:

$$x_{i,k} = 0, \quad \forall u_k \notin \mathcal{F}_i$$

7.3 Instance load and objective

每个 instance 的 load 为:

$$L_k = \sum_{i=1}^N x_{i,k} l_{i,k}$$

Batch latency / makespan 为:

$$T_{batch} = \max_{u_k \in \mathcal{U}} L_k$$

优化目标为:

$$\min_{\mathbf{x}} T_{batch} = \min_{\mathbf{x}} \max_{u_k \in \mathcal{U}} \sum_{i=1}^N x_{i,k} l_{i,k}$$

subject to:

$$\sum_{k=1}^K x_{i,k} = 1, \quad \forall i$$

$$x_{i,k} = 0, \quad \forall u_k \notin \mathcal{F}_i$$

7.4 问题类型

该问题是 quality-constrained unrelated parallel machine scheduling with eligibility constraints, 可视为 $R|eligibility|C_{max}$ 变体。该类问题是 NP-hard, 因此 QMAR 不 claim 求全局最优, 而是实现一个 lightweight request-ranking-based greedy scheduler。

8 QMAR 算法设计

8.1 算法定位

QMAR 表示 Quality-Margin-Aware Routing。更准确地说, 它是:

QMAR is a request-ranking-based greedy scheduler for quality-constrained min-max VLM instance assignment.

它先估计 request-specific suitability 与 answer-type-aware latency, 再对 request 排序, 最后进行 load-balanced assignment。

8.2 算法输入

QMAR 的输入包括:

- request batch $\mathcal{B}$;
- instance pool $\mathcal{U}$;
- model suitability scores $s_{i,m}$;
- predicted answer type $\hat{a}_i$;

- latency table $\bar{l}_{k,a}$;
- communication profile $l_{i,k}^{comm}$;
- quality threshold τ_i .

8.3 Step 1: Feasible set construction

对每个 request 构造:

$$\mathcal{F}_i = \{u_k \in \mathcal{U} \mid s_{i,m(k)} \geq \tau_i\}$$

同时计算:

$$l_{i,k} = \bar{l}_{k,\hat{a}_i} + l_{i,k}^{comm}$$

8.4 Step 2: Request ranking

QMAR 先调度最难安排的 requests。对每个 request 计算三个 ranking feature。

8.4.1 Feasible set size

$$q_i = |\mathcal{F}_i|$$

q_i 越小, 说明 request 可选 instance 越少, 越应该优先。

8.4.2 Quality margin

$$\delta_i = \max_{u_k \in \mathcal{F}_i} (s_{i,m(k)} - \tau_i)$$

δ_i 越小, 说明 request 的质量余量越小, 越脆弱, 越应该优先。

8.4.3 Minimum feasible latency

$$p_i = \min_{u_k \in \mathcal{F}_i} l_{i,k}$$

p_i 越大, 说明 request 即使在最快可行 instance 上也较慢, 越应该优先。

8.4.4 Lexicographic sorting rule

排序规则为:

```
Sort requests by:
1. smaller feasible set size  $q_i$ ;
2. smaller quality margin  $\delta_i$ ;
3. larger minimum feasible latency  $p_i$ .
```

等价地, 可将排序 key 写为:

$$\rho_i = (q_i, \delta_i, -p_i)$$

并按 ρ_i 升序排序。

8.5 Step 3: Load-balanced greedy assignment

初始化每个 instance 当前 load:

$$L_k = 0, \quad \forall k$$

按排序后的 request 顺序依次处理。对 request r_i , 选择使分配后负载最小的 feasible instance:

$$u^* = \arg \min_{u_k \in \mathcal{F}_i} (L_k + l_{i,k})$$

然后更新:

$$x_{i,u^*} = 1$$

$$L_{u^*} \leftarrow L_{u^*} + l_{i,u^*}$$

最终输出 assignment 与 batch latency:

$$T_{batch} = \max_k L_k$$

8.6 Fallback rule

如果某个 request 没有任何满足 threshold 的 instance:

$$\mathcal{F}_i = \emptyset$$

则触发 fallback。第一版推荐选择 predicted suitability 最高的 instance:

$$u^* = \arg \max_{u_k \in \mathcal{U}} s_{i,m(k)}$$

该 assignment 需要标记为 quality-risk assignment。评估时统计 fallback rate 与 quality violation rate。

8.7 Pseudocode

Algorithm: QMAR Batch Scheduling

Input:

batch B, instance pool U, suitability scores s,
predicted answer types $\hat{a}$, latency table, communication profile,
quality thresholds τ

1. For each request r_i in B:
 - For each instance u_k in U:
 - map instance to model $m(k)$
 - compute suitability $s_{i,k} = s_{i,m(k)}$
 - estimate latency $l_{i,k} = \text{latency_table}[k][\hat{a}_i] + \text{comm_cost}[i][k]$
 - $F_i = \{u_k \mid s_{i,k} \geq \tau_i\}$
 - If F_i is empty:
 - mark r_i as fallback candidate
 - define $F_i = U$ for scheduling with fallback flag
2. For each request r_i :
 - $q_i = |F_i|$ if not fallback else +infinity or special group
 - $\delta_i = \max(s_{i,k} - \tau_i)$ over original feasible set
 - $p_i = \min(l_{i,k})$ over F_i
3. Sort requests by:

```

    q_i ascending, delta_i ascending, p_i descending
4. Initialize  $L_k = 0$  for all instances
5. For each request  $r_i$  in sorted order:
    If original  $F_i$  is not empty:
         $u^* = \operatorname{argmin}_{\{u_k \in F_i\}} (L_k + l_{i,k})$ 
    Else:
         $u^* = \operatorname{argmax}_{\{u_k \in U\}} s_{i,k}$ 
        mark assignment as quality-risk
    assign  $r_i$  to  $u^*$ 
     $L_{u^*} = L_{u^*} + l_{i,u^*}$ 
6. Return assignment, instance loads,  $T_{\text{batch}} = \max_k L_k$ ,
    fallback flags and routing logs

```

8.8 Complexity

构造 feasible set 需要 $O(NK)$ 。排序需要 $O(N \log N)$ 。贪心分配最坏需要 $O(NK)$ 。总复杂度为：

$$O(NK + N \log N)$$

该复杂度适合 batch-level runtime scheduler。

9 Implementation Components for Claude Code

9.1 建议模块划分

实现时建议按以下模块组织，避免把数据处理、预测器、调度器和评估逻辑混在一起。

Module	Responsibility
data_adapter	读取 VL-RouterBench / MMR-Bench，转成统一 RequestRecord schema。
profiler	构造 quality profile、answer type label、latency table、communication profile。
predictor	实现 frozen image/text encoder、fusion MLP、suitability head、answer-type head。
latency_estimator	根据 predicted answer type、instance profile、communication profile 生成 $l_{i,k}$ 。
qmar_scheduler	实现 feasible set、request ranking、greedy assignment、fallback logging。
evaluator	计算 quality-side、system-side、routing-side metrics。
experiments	管理不同 dataset、threshold、baseline、ablation 的实验入口。
configs	保存 dataset、model pool、instance pool、threshold、latency profile、training hyperparameters。

9.2 推荐目录结构

```

qmar_project/
  data_adapters/
    vl_routerbench_adapter.py
    mmr_bench_adapter.py
    common_schema.py
  profiling/

```

```

    build_quality_profile.py
    build_answer_type_labels.py
    build_latency_table.py
    communication_profile.py
predictor/
    encoders.py
    fusion.py
    model.py
    train.py
    infer.py
scheduler/
    latency_estimator.py
    qmar.py
    baselines.py
evaluation/
    metrics.py
    run_eval.py
    plots.py
configs/
    dataset.yaml
    model_pool.yaml
    instance_pool.yaml
    qmar.yaml
outputs/
    profiles/
    checkpoints/
    assignments/
    results/

```

这只是实现组织建议，不要求文件名完全一致。

10 Baselines and Ablations

10.1 必要 baselines

至少实现以下 baseline：

Baseline	Description
Random feasible	在 feasible instances 中随机选择。
Fastest feasible	在 feasible instances 中选择 estimated latency 最小的 instance。
Highest suitability	选择 predicted suitability 最高的 instance，不主要考虑 load balance。
Latency-only greedy	不使用 quality-margin ranking，只按 load-balanced greedy 分配。
QMAR w/o answer type	使用 instance 平均 latency，不使用 answer-type-aware latency table。
QMAR full	使用 suitability、answer type、quality-margin ranking 与 greedy load balancing。

10.2 可选 oracle baseline

小规模 batch 可以用 ILP 或 exhaustive search 构造 oracle makespan，用于验证 QMAR 与最优解之间的 gap。但不要把 ILP 作为主算法。

11 Evaluation Metrics

11.1 Quality-side metrics

- average accuracy;
- quality satisfaction rate;
- threshold violation rate;
- fallback rate;
- oracle regret.

11.2 System-side metrics

- batch latency / makespan;
- average request latency;
- P95 request latency;
- throughput;
- instance load imbalance.

11.3 Routing-side metrics

- selected model / instance distribution;
- feasible set size distribution;
- quality margin distribution;
- fallback case analysis;
- load distribution across instances.

12 Recommended Experiment Sequence

12.1 Phase 1: VL-RouterBench main pipeline

先用 VL-RouterBench 跑通完整流程：

1. 转换数据为统一 RequestRecord schema。
2. 构造 model correctness labels。
3. 构造 answer type / length bucket labels。
4. 构造或模拟 instance latency table。
5. 训练 Predictor。
6. 在 test split 上生成 suitability 与 answer type prediction。
7. 运行 QMAR 与 baselines。
8. 输出 quality-latency trade-off results。

12.2 Phase 2: MMR-Bench validation

使用 MMR-Bench 重复同样流程，但作为外部验证。不要直接与 VL-RouterBench 混合训练。可以做：

- retrain-on-MMR evaluation;
- train-on-VL, test-on-MMR common subset evaluation;
- protocol sensitivity analysis。

12.3 Phase 3: Ablation

重点验证两个核心设计是否有效：

- answer-type-aware latency estimation 是否降低 batch latency;
- quality-margin-aware request ranking 是否减少 fallback、降低 makespan 或提升质量约束满足率。

13 Expected Outputs

Claude Code 最终应生成以下实验产物：

```
outputs/profiles/  
  quality_profile.csv  
  answer_type_labels.csv  
  latency_table.csv  
  
outputs/checkpoints/  
  predictor_best.pt  
  
outputs/assignments/  
  qmar_assignments_<dataset>_<tau>.csv  
  baseline_assignments_<dataset>_<tau>.csv  
  
outputs/results/  
  summary_metrics.csv  
  threshold_sweep.csv  
  ablation_results.csv  
  fallback_cases.csv  
  load_balance_stats.csv
```

每个 assignment 结果至少包含：

```
request_id  
assigned_instance  
assigned_model  
predicted_suitability  
predicted_answer_type  
estimated_latency  
is_quality_feasible  
is_fallback  
current_instance_load_after_assignment
```

14 Implementation Notes

14.1 关键设计约束

- Predictor 输出 model-level suitability，不直接输出 instance-level suitability。

- QMAR 在 runtime 中将 model suitability 映射到 instances。
- Latency profile 必须是 instance-level。
- Answer type 第一版使用 short / medium / long，避免复杂语义标注。
- Fallback assignment 必须被显式标记，不能混入正常 feasible assignment。
- Evaluation 中需要同时报告 quality 与 system latency，不能只报告 accuracy。

14.2 最终一句话定位

Predictor characterizes each image-text request by estimating model suitability and answer-type-dependent workload. QMAR then performs quality-constrained, request-ranking-based greedy assignment to minimize batch makespan over heterogeneous VLM instances.